Lab 3: Serve and measure

Measuring what your model costs to run

Aarne Talman

12 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	**10 / 12 Nov**	**Inference**	**Lab 3: serve**	**Aarne**
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

The question

The question today

Today you put a language model behind a server and measure what it costs to run. You may use the adapter from Lab 2 or another compatible model.

Today

What does it cost to run, and what do you give up to make it cheaper?

You will produce

- one **speed-versus-quality table**
- a **deployment recommendation**, 3–5 sentences
- a **one-command restart script** you can reuse in later labs

You will not

- train anything
- change your prompt
- change your held-out set

Every number today is a measurement on a model you already have.

In your final report, the deployment recommendation must be supported by the table. Start with `lab3-example/README.md`.

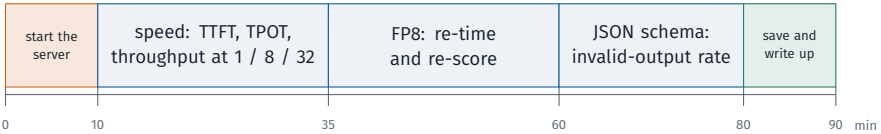
Material for the final report

Configuration	TTFT ms, p50 / p95	TPOT ms, p50 / p95	Output tok/s @ 1 / 8 / 32	Task score	Invalid %
bf16 + LoRA adapter	–	–	–	–	–
FP8 + LoRA adapter	–	–	–	–	–
bf16 + JSON schema	–	–	–	–	–

TTFT: how long until the *first* token appears. **TPOT**: milliseconds per token after that. **p50 / p95**: the median request, and the slowest one in twenty. **@ 1 / 8 / 32**: with that many requests in flight at the same instant. **Invalid %**: outputs your parser could not use.

Same model, same adapter, same held-out set, same prompts, same decoding parameters in every row. **One variable changes per row.** Task score comes from the **same held-out scorer used in week 2**, unchanged.

Ninety minutes



If the server is not up by **minute 15**, stop debugging alone and put your hand up. Everything after that block depends on it.

Serving

Measure honestly, 1: warm up, then fix the length

Before you time anything

- send ≈ 20 throwaway requests and **discard them**
- then start recording

Fix everything except the one variable

- `max_tokens=128` and `ignore_eos=True`, so that every request emits exactly 128 tokens
- the same prompt set, in the same order
- `temperature=0.0` everywhere

Without a fixed output length you are measuring *how verbose the model is* rather than how fast the server is. FP8 changes verbosity.

The first request is always slow

One-off work all lands on request number one: the GPU compiles and tunes its kernels, the weights are read in for real, and nothing is cached yet. Without a warm-up you are measuring **startup**.

The opposite trap: the server remembers the work done for a prompt it has already seen (“prefix caching”), so the *second* identical prompt looks artificially fast. Vary your prompts, or serve with `--no-enable-prefix-caching` while benchmarking.

Measure honestly, 2: TTFT, TPOT, concurrency

Number	What it is	Whose experience
TTFT	waiting in the queue, then reading your prompt, until the first token leaves	the user staring at a blank box
TPOT	milliseconds per token once it is going	the user reading the answer as it streams
Throughput	total output tokens/s across all requests at once	nobody; it is the *operator's* number

In words: the user's total wait is the time to the first token, plus the time per token times the number of tokens.

Sweep concurrency at 1 / 8 / 32, meaning that many requests in flight at the same instant. Expect throughput to rise while TPOT gets worse, which is Tuesday's latency–throughput curve drawn with your own numbers.

Sanity check at concurrency 1: every token re-reads the whole model out of GPU memory, so tokens/s cannot beat about 500/s for an 8 GB model on a GH200.

Source: Definitions and the roofline estimate as given in Tuesday's lecture, section 2; the latency–throughput curve, section 4.

Measure honestly, 3: report the distribution

Record **one row per request**, not an average:

- request id, concurrency level, configuration
- TTFT, TPOT, output tokens, wall-clock start

Then report **median (p50) and p95** for each cell of the table, from **at least 50 requests**. State how many. The production version of this is

goodput: requests per second that meet a latency target you choose and state, say first token under 500 ms **and** under 50 ms per token after that. Those thresholds are only an example.

Pick your own and justify them from your task.

The mean hides the tail users complain about

A mean latency over a batch of requests is dominated by the many fast ones, so it tells you nothing about the slow few. A server with a good mean and a terrible p95 still feels broken to use.

Keep the raw per-request log in the repo. The report shows p50 and p95, and the log is where anyone can check them.

QUALITY

Score the quantised model yourself

Run the **same held-out scorer used in week 2** against the FP8 endpoint. It should take a model or a base URL and return the task score in one call.

Same held-out set, same prompts, same decoding parameters as the bf16 score. Otherwise you are comparing two different experiments.

Where quantisation damage actually shows up

- long inputs
- non-English text
- long reasoning chains

Headline benchmarks are the *last* place it appears, which is why a published "FP8 is lossless" claim says nothing about your task.

Report both halves

Gained: tokens/s, p95 latency, memory freed for KV cache, concurrency reached.

Paid: score points on the held-out set.

If you report only one of these halves, you have written an advert.

Look at your **ten worst examples** under FP8, not only the aggregate. If the aggregate moved by less than the noise of your metric, say so.

Source: Quality cost of quantisation as discussed in Tuesday's lecture, section 5. The score change on your task is a measurement, not a citation.

Measure the invalid-output rate, both ways

The protocol

1. A fixed set of held-out prompts, asking for JSON in the prompt text, **no** schema. Parse each output. Count failures.
2. The *same* prompts with `guided_json`. Count again.
3. Score *content* in both runs with the same held-out scorer.

Invalid means: not parseable, or parseable but missing a required field, or a label outside your enum. Say which definition you used.

Record two numbers

Syntax: invalid-output rate, with and without the schema.

Content: task score, with and without the schema.

Forcing valid syntax can cost content quality: the model gets pushed onto token paths it would not have chosen. If your invalid rate goes to zero and your task score drops two points, **that is the finding**, and it belongs in the recommendation.

Constrained decoding also costs a little speed. Time it if you have the minutes; the score matters more.

Bonus: temperature 0, alone and in a crowd

The procedure

1. Pick one prompt. `temperature=0`, `max_tokens=256`.
2. Send it **alone** on an idle server. Save the output.
3. Send it again alone. Confirm it is identical.
4. Send the *same* prompt as one of **64 concurrent** requests. Save that output.
5. Diff against the solo output.
6. Repeat steps 4–5 twenty times. Report how many of the 20 differ.

Why it happens

In floating point, $(a + b) + c$ and $a + (b + c)$ can differ in the last bits, and the GPU adds numbers up in a different order depending on how many requests it is batching. So *other people's requests* can change your scores, and occasionally the words you get back.

The fix is called **batch invariance**. This is not GPU randomness and it is not a seed problem; see Tuesday, section 8.

One line in the report: what this means for reproducible evaluation.

Admin

Files and results to keep

In the repo

- `results/speed-quality.csv`, the table with one row per configuration
- `results/requests.csv`, the raw per-request log
- `scripts/serve.sh`, tested from a cold start
- the notebook with the timing harness
- a `README` giving the base URL, port, model and adapter names

In the report, section 4

The table, and 3–5 sentences of recommendation.

The recommendation must answer

Which configuration would you deploy, and under what latency target?

What did FP8 cost you, in score points?

What would you change at ten times the traffic?

There is no intermediate submission. Keep these files for the final repository and reuse the endpoint in later labs if it suits your project.

Rubric reminder: Results 30 %, Analysis 30 %. The table counts as Results and the recommendation as Analysis.

GPU ETIQUETTE

- **Never serve from a login node.** No GPU there, and the CPUs are shared by everyone on Roihu.
- `gputest` (15 min) before `gpumedium` (36 h) for anything you submit as a batch job.
- A `gpuinteractive` session holds a whole GH200 for up to 12 hours. **Close it when you leave today**, from the Roihu web interface rather than by closing the browser tab.
- Stop your vLLM server before you stop the session, so the port and the memory are released cleanly.

Why it matters more this week

We share one course project allocation, and **weeks 5 and 6 need this endpoint again**. Every forgotten session today is GPU time missing from Dmitry's labs.

Your server dies with the allocation. That is expected, and it is why `serve.sh` has to work from a cold start.

`squeue --me` to see what you are holding.
`scancel <jobid>` to let it go.

On the wall for the rest of the session

Timeline

min	
0–10	server up, adapter visible
10–35	speed: 1 / 8 / 32
35–60	FP8: re-time, re-score
60–80	JSON schema: invalid rate
80–90	save results and write up

Server not up by minute 15? Hand up.
Bonus (determinism) only if rows 1–3 are done.

Done when

- [ ] server answers on `:8000`, model `task`
- [ ] warm-up requests discarded
- [ ] p50 *and* p95 for TTFT and TPOT, at 1 / 8 / 32
- [ ] number of requests recorded for every cell
- [ ] FP8 row: timed *and* scored
- [ ] invalid-output rate, with and without schema
- [ ] `results/speed-quality.csv` saved
- [ ] `scripts/serve.sh` tested from a clean shell
- [ ] 3–5 sentence deployment recommendation
- [ ] GPU session closed

Keep these outputs for the final report and code archive.

When it breaks

Symptom	First thing to try
OOM when the server starts	Something else holds the GPU, usually a week 2 notebook kernel. Restart the kernel, then run `nvidia-smi`, which lists what is holding the card. Then lower `--gpu-memory-utilization` to 0.7 and `--max-model-len` to 2048.
`Address already in use`	Your previous server is still running. `pkill -f "vllm serve"`, or use `--port 8001` and change the base URL to match.
404, or the adapter has no effect	`model=` must be the name on the *left* of `--lora-modules name=path`, not the repo id. Check `--enable-lora` is present and the path contains `adapter_config.json`.
Fluent nonsense, normal latency	Tokenizer or chat-template mismatch: the adapter was trained on the instruct model and you are serving the base one, or the reverse. Serve the checkpoint you fine-tuned.
Server dies when the allocation ends	Expected: the server is a child of your `gpuinteractive` session. Start a new session and run `bash scripts/serve.sh`.